

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
FACULTAD DE CIENCIAS DE LA COMPUTACIÓN
CARRERA DE SOFTWARE



TEMA:

Comparación entre modelos de proceso de requisitos en escenarios reales

MATERIA:

Ingeniería en Requerimientos

DOCENTE:

Ing. Guerrero Ulloa Gleiston Ciceron

ESTUDIANTES:

Calle Delgado Kamila Annabella

Macias Herrera Josthyn Esteban

Perez Ruiz Carlos Andres

CURSO:

Cuarto semestre, paralelo "A"

AÑO LECTIVO:

2026-2027

Comparación entre modelos de proceso de requisitos en escenarios reales

En el desarrollo de software, uno de los aspectos más importantes es entender correctamente lo que necesita el cliente o usuario. A este proceso se le conoce como proceso de requisitos, y consiste en recopilar, analizar, documentar y validar todas las necesidades que tendrá el sistema antes de ser construido.

A lo largo del tiempo se han creado diferentes modelos para gestionar estos requisitos. Cada uno tiene formas distintas de trabajar, organizar las tareas y comunicarse con el cliente. Algunos modelos son más rígidos y ordenados, mientras que otros son más flexibles y permiten cambios constantes.

La elección del modelo adecuado depende del tipo de proyecto, el tiempo disponible, el presupuesto, la complejidad del sistema y el nivel de participación del cliente. Por ejemplo, no es lo mismo desarrollar una aplicación móvil para una pequeña empresa que un sistema médico o bancario donde cualquier error podría causar problemas graves.

A continuación, se presenta una comparación detallada de cuatro modelos de proceso de requisitos muy utilizados en escenarios reales [1]

1. Modelo Tradicional en Cascada

El modelo en cascada es uno de los métodos más antiguos y conocidos dentro de la ingeniería de software. Su funcionamiento es parecido a una cascada de agua: cada etapa cae sobre la siguiente y no se puede regresar fácilmente atrás. En este modelo, primero se recopilan todos los requisitos del sistema y se documentan detalladamente. Después de eso comienzan las demás fases, como el diseño, programación, pruebas y mantenimiento. La idea principal es que todo esté claramente definido desde el inicio para evitar cambios durante el desarrollo. Por eso se considera un modelo muy estructurado y organizado.

Este modelo suele utilizarse en proyectos donde los requisitos ya están muy claros y casi no cambian con el tiempo. Por ejemplo, es común en sistemas bancarios, plataformas gubernamentales o programas administrativos donde existen reglas fijas y procesos bien definidos. En estos casos, las empresas necesitan mucha documentación y control para evitar errores, especialmente porque cualquier fallo puede afectar información importante o procesos legales.

Una de las mayores ventajas del modelo en cascada es que permite llevar un orden muy claro en cada etapa del proyecto. Todo está planificado desde el inicio, lo que facilita calcular tiempos, costos y recursos necesarios. Además, al existir bastante documentación, el mantenimiento del sistema en el futuro resulta más sencillo. Sin embargo, también tiene algunas desventajas importantes. Si el cliente desea cambiar algo después de iniciar el desarrollo, realizar modificaciones puede ser complicado y costoso. Además, muchas

veces el usuario no puede ver el sistema funcionando hasta etapas muy avanzadas, lo que aumenta el riesgo de descubrir errores demasiado tarde [2].

2. Modelo Ágil Scrum

El modelo ágil Scrum nació como una forma más flexible y rápida de desarrollar software. A diferencia del modelo en cascada, Scrum entiende que en muchos proyectos los requisitos pueden cambiar constantemente y que eso forma parte natural del proceso. En lugar de desarrollar todo el sistema de una sola vez, el trabajo se divide en pequeñas etapas llamadas Sprint, que suelen durar pocas semanas.

Durante cada sprint, el equipo desarrolla una parte funcional del sistema y luego la presenta al cliente para recibir comentarios y sugerencias. Gracias a esto, el proyecto puede mejorar continuamente y adaptarse a nuevas necesidades. Los requisitos no se definen completamente desde el inicio, sino que van evolucionando conforme avanza el trabajo y aparecen nuevas ideas o cambios solicitados por los usuarios.

Este modelo es muy utilizado en aplicaciones móviles, páginas web, videojuegos y empresas tecnológicas donde todo cambia rápidamente. Por ejemplo, una aplicación de compras en línea necesita actualizar constantemente sus funciones según las preferencias de los usuarios y las nuevas tendencias del mercado. Scrum permite hacer estos cambios sin detener completamente el proyecto.

Entre sus ventajas destaca la rapidez para adaptarse a cambios y la comunicación constante entre el cliente y el equipo de desarrollo. Además, el usuario puede ver resultados en poco tiempo, lo que genera mayor confianza y satisfacción. También ayuda a detectar errores rápidamente y mejora el trabajo en equipo. Sin embargo, Scrum también puede generar problemas si no existe una buena organización o si el cliente no participa activamente en el proyecto.

3. Modelo Espiral

El modelo espiral es una metodología que combina organización, desarrollo progresivo y análisis de riesgos. Su nombre proviene de la forma en que funciona el proyecto, ya que cada etapa se desarrolla en ciclos que van creciendo como una espiral. En cada vuelta se analizan los requisitos, se identifican riesgos, se desarrolla una parte del sistema y luego se evalúan los resultados antes de continuar.

La principal característica de este modelo es que busca reducir riesgos desde las primeras etapas del proyecto. Esto significa que antes de avanzar se revisan posibles problemas técnicos, económicos o de seguridad para evitar errores graves en el futuro. Gracias a esto, el sistema puede mejorar continuamente mientras avanza el desarrollo.

En escenarios reales, este modelo se utiliza principalmente en proyectos muy complejos o delicados, como sistemas médicos, programas militares, software aeronáutico o plataformas científicas. Por ejemplo, en un sistema hospitalario cualquier error podría afectar

la atención de los pacientes, por lo que es necesario revisar constantemente la seguridad y el correcto funcionamiento del software [3]

Una de las ventajas más importantes del modelo espiral es que permite detectar problemas temprano y realizar mejoras continuas. Además, ofrece mucho control sobre el proyecto y ayuda a desarrollar sistemas más seguros.

4. Modelo Basado en Prototipos

El modelo basado en prototipos se enfoca en crear versiones preliminares del sistema antes de desarrollar el producto final. Estas versiones, llamadas prototipos, sirven para que el cliente pueda observar cómo funcionará el sistema y así expresar mejor sus ideas, necesidades y sugerencias. Muchas veces las personas no tienen completamente claro lo que desean hasta que ven algo funcionando, y este modelo ayuda precisamente a resolver ese problema.

El proceso normalmente comienza con un diseño sencillo que incluye las funciones más importantes del sistema. Luego el cliente prueba el prototipo y da su opinión sobre cambios o mejoras necesarias. Después, el equipo realiza modificaciones y vuelve a mostrar una nueva versión mejorada. Este ciclo puede repetirse varias veces hasta que los requisitos estén suficientemente claros para desarrollar el sistema definitivo.

Este modelo es muy utilizado en aplicaciones móviles, plataformas educativas, sistemas empresariales y programas donde la experiencia del usuario es muy importante. Por ejemplo, una universidad que quiere crear un aula virtual puede necesitar varios prototipos antes de decidir cuál diseño resulta más cómodo y fácil de usar para estudiantes y profesores.

La principal ventaja de este modelo es que mejora mucho la comunicación entre el cliente y los desarrolladores. Además, ayuda a descubrir errores o necesidades que inicialmente no se habían considerado. También permite que el usuario participe activamente en el proyecto y se sienta más satisfecho con el resultado final [4].

Modelo	Característica principal	Ventaja	Desventaja
Cascada	Trabajo secuencial y ordenado.	Mucha organización y documentación.	Poco flexible ante cambios.
Scrum	Desarrollo rápido y flexible por sprints.	Se adapta fácilmente a cambios.	Requiere participación constante del cliente.
Espiral	Desarrollo basado en análisis de riesgos.	Mayor seguridad y control.	Alto costo y complejidad.
Prototipos	Creación de versiones preliminares.	Ayuda a entender mejor los requisitos.	Puede aumentar tiempo y costos.

- [1] V. Guntamukkala, H. J. Wen, and J. M. Tarn, "An empirical study of selecting software development life cycle models," *Human Systems Management*, vol. 25, no. 4, pp. 265–278, 2006, doi: 10.3233/HSM-2006-25405;PAGE:STRING:ARTICLE/CHAPTER.
- [2] A. Saravanos and M. X. Curinga, "Simulating the Software Development Lifecycle: The Waterfall Model," *Applied System Innovation 2023, Vol. 6, Page 108*, vol. 6, no. 6, p. 108, Nov. 2023, doi: 10.3390/ASI6060108.
- [3] R. Sari, A. M. Rifa'i, M. S. Ahsan, M. R. Pahlevi, and M. I. Arief, "The Systematic Literature Review of the spiral development model: Topics, trends, and application areas," *International Journal of Research and Applied Technology (INJURATECH)*, vol. 2, no. 2, pp. 154–171, Nov. 2022, doi: 10.34010/INJURATECH.V2I2.8372.
- [4] N. Cerpa and J. Verner, "Prototyping: some new results," *Inf. Softw. Technol.*, vol. 38, no. 12, pp. 743–755, Jan. 1996, doi: 10.1016/0950-5849(96)01098-1.